

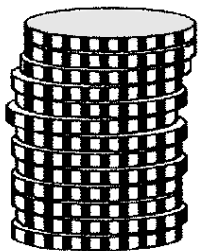
Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

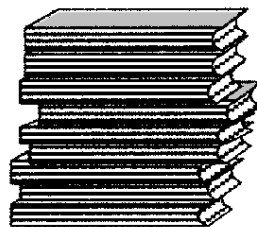
[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)

3.3 STACKS

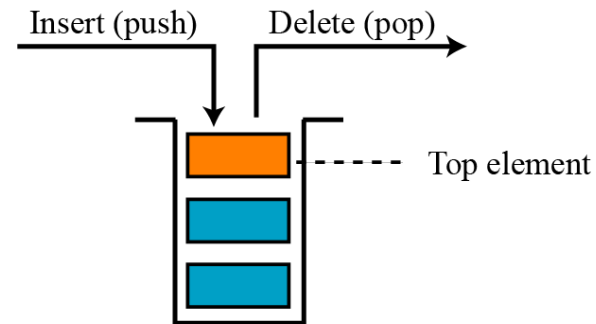
- A stack is a restricted linear list in which all additions and deletions are made at one end, the top.
- Stacks are known as **Last In, First Out (LIFO)** data structures.



Stack of coins



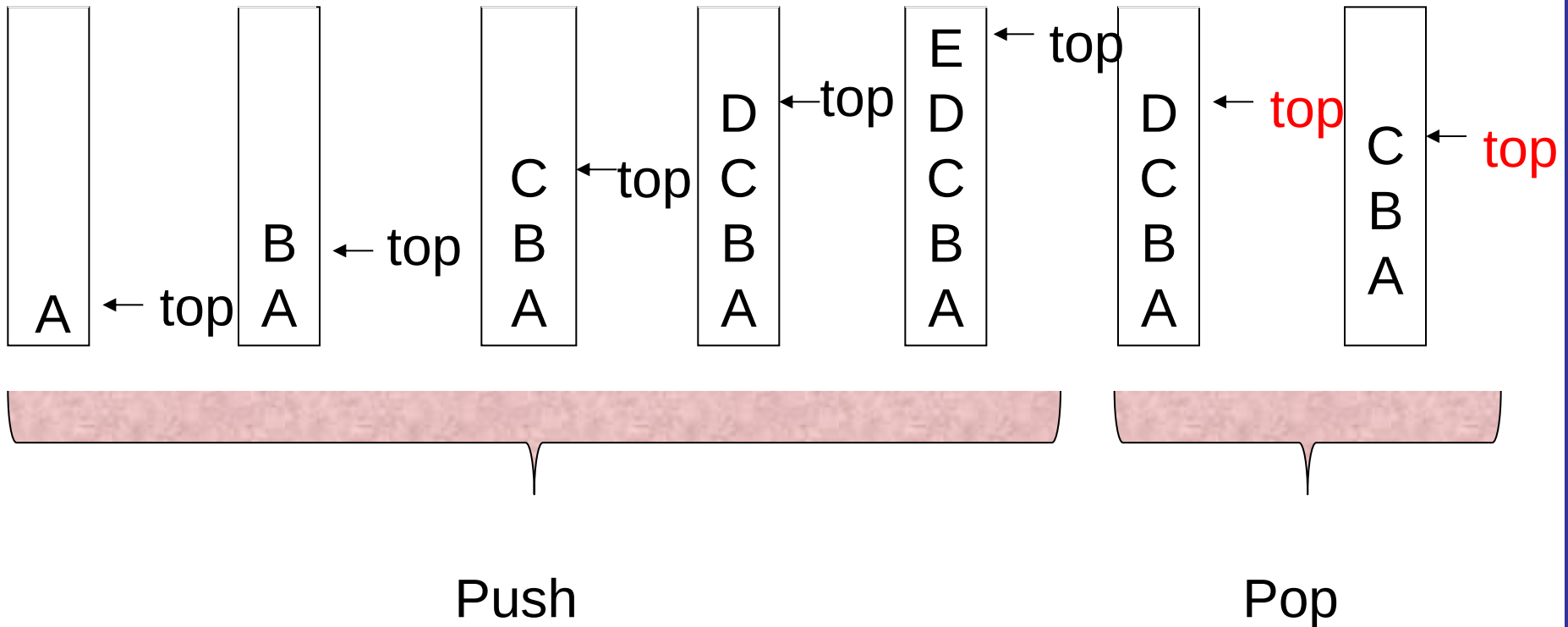
Stack of books



Computer stack

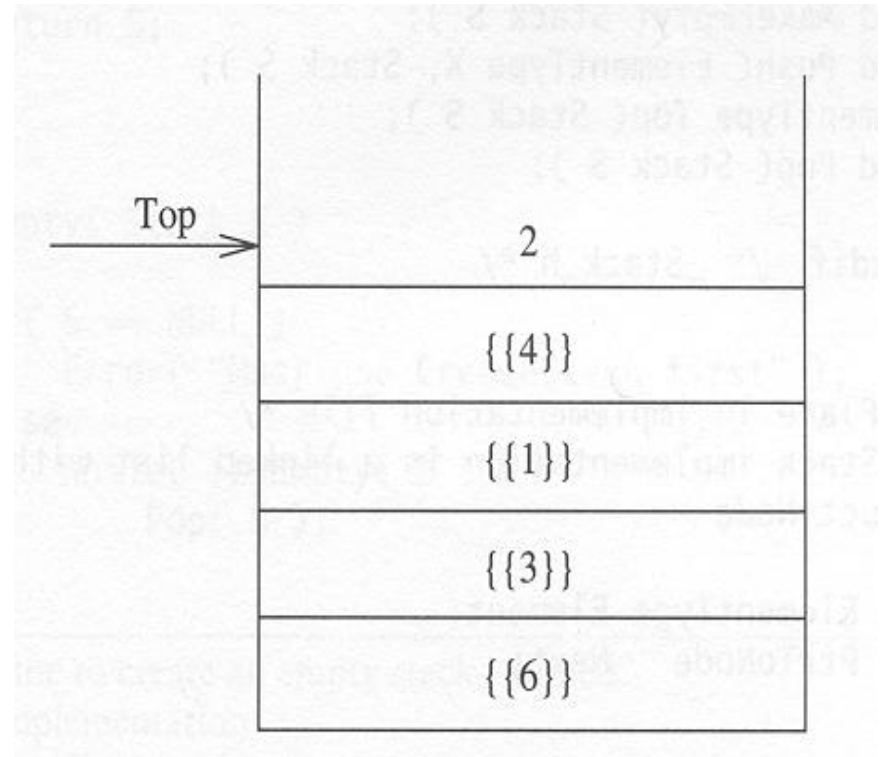
Three representations of stacks

Last In First Out (LIFO)



Stacks ADT

- LIFO structure
- Access from the top
- Basic Operations
 - empty (s)
 - pop (s)
 - push (s, i)
 - top (s)



Implementation of Stacks using Linked List

```
struct Node;  
typedef struct Node *node_ptr;  
typedef node_ptr Stack;
```

```
struct Node  
{ ElementType Element;  
  node_ptr Next;  
}
```

Implementation of Stacks

```
int IsEmpty(Stack S);  
Stack CreatStack(void);  
void DisposeStack(Stack S);  
void MakeEmpty(Stack S);  
void Push(ElementType X, Stack S);  
void Pop(Stack S);  
ElementType Top(Stack S);
```

Implementation of Stacks

- Test for an empty stack

```
int IsEmpty(Stack S)
{
    return (S→Next == NULL)
}
```

Implementation of Stacks

- Create an empty stack

```
STACK create_stack( void )
{
    STACK S;
    S = (STACK) malloc( sizeof( struct node ) );
    if( S == NULL )
        fatal_error("Out of space!!!");
    return S;
}
```


Implementation of Stacks

- Make NULL

```
void make_null( STACK S )  
{  
    if( S != NULL )  
        S->next = NULL;  
    else  
        error("Must use create_stack first");  
}
```

Implementation of Stacks

■ Push onto a stack

```
void push( element_type x, STACK S )
{
    node_ptr tmp_cell;
    tmp_cell = (node_ptr) malloc( sizeof ( struct node ) );
    if( tmp_cell == NULL )
        fatal_error("Out of space!!!");
    else
    {
        tmp_cell->element = x;
        tmp_cell->next = S->next;
        S->next = tmp_cell;
    }
}
```

Implementation of Stacks

- Return top element in a stack

```
element_type top( STACK S )  
{  
    if( is_empty( S ) )  
        error("Empty stack");  
    else  
        return S->next->element;  
}
```

Implementation of Stacks

■ Pop from a stack

```
void pop( STACK S )
{
    node_ptr first_cell;
    if( is_empty( S ) )
        error("Empty stack");
    else
    {
        first_cell = S->next;
        S->next = S->next->next;
        free( first_cell );
    }
}
```

Algebraic Expressions

- Infix expressions
 - An operator appears between its operands
 - Example: $a + b$
- Prefix expressions
 - An operator appears before its operands
 - Example: $+ a b$
- Postfix expressions
 - An operator appears after its operands
 - Example: $a b +$

Algebraic Expressions

- Infix expressions:

$$a + b * c + (d * e + f) * g$$

- Postfix expressions:

$$abc*+de*f+g*+$$

- Prefix expressions:

$$++a*bc*+*defg$$

Algebraic Expressions

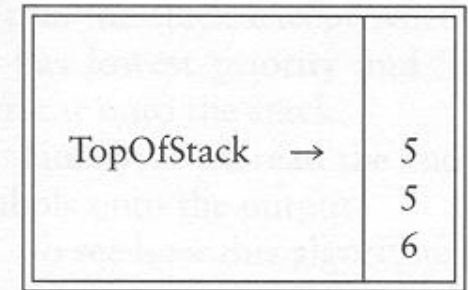
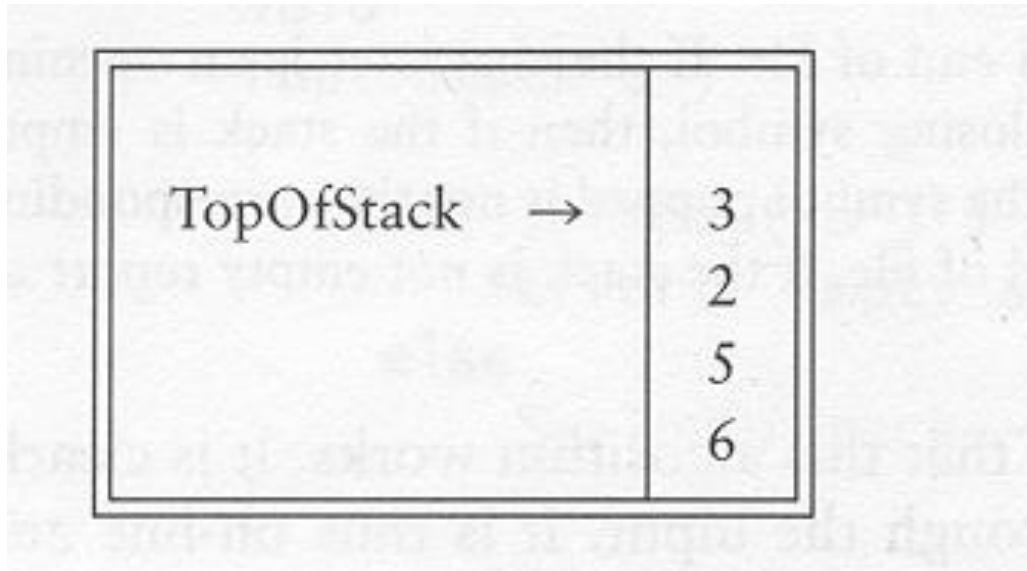
- To convert a fully parenthesized infix expression to a prefix form
 - Move each operator to the position marked by its corresponding open parenthesis
 - Remove the parentheses
 - Example
 - Infix expression: $((a + b) * c)$
 - Prefix expression: $* + a b c$

Algebraic Expressions

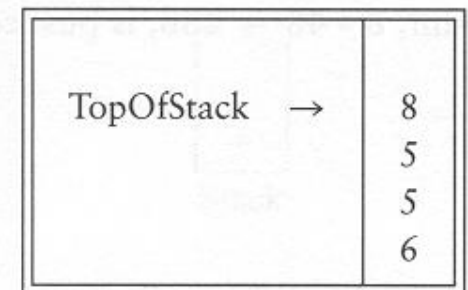
- To convert a fully parenthesized infix expression to a postfix form
 - Move each operator to the position marked by its corresponding closing parenthesis
 - Remove the parentheses
 - Example
 - Infix form: $((a + b) * c)$
 - Postfix form: $a b + c *$

Example 1 (Calculator)

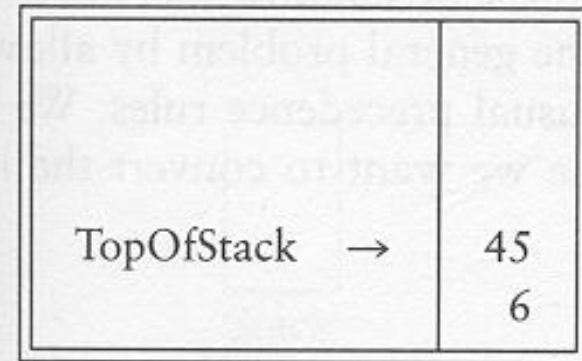
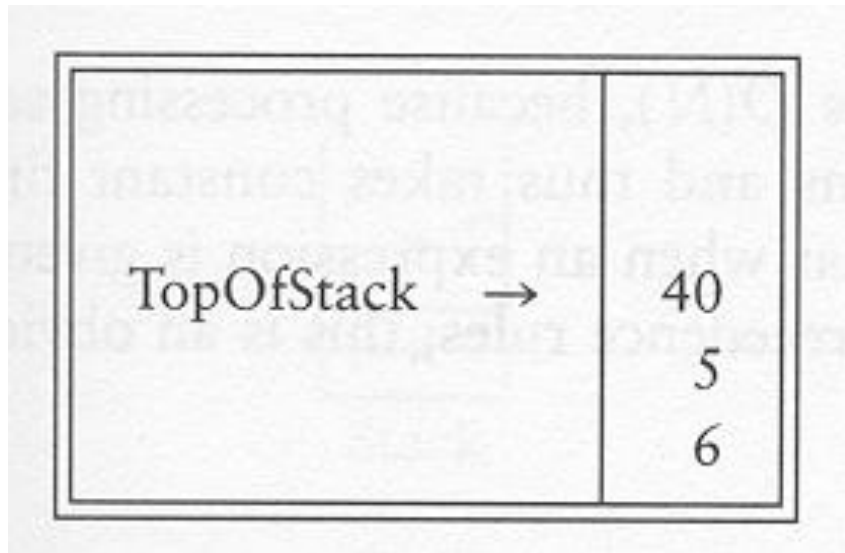
- How does the postfix work?
- e.g. $6\ 5\ 2\ 3\ +\ 8\ *\ +\ 3\ +\ *$



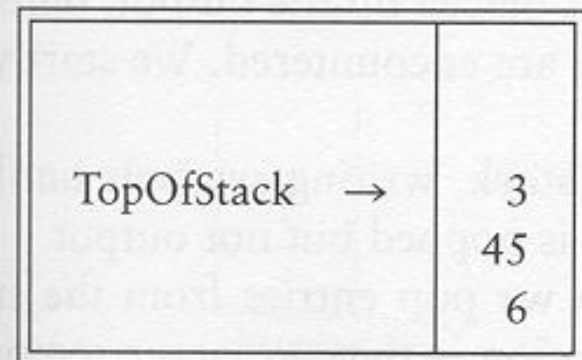
Next 8 is pushed.



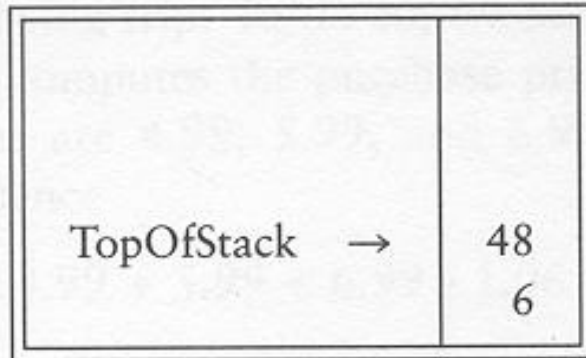
Example 1 (Calculator)



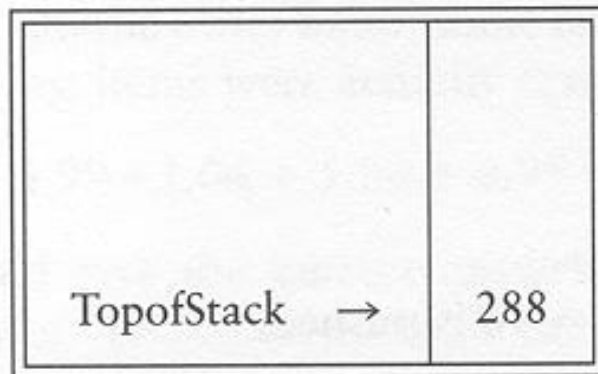
Now, 3 is pushed.



Example 1 (Calculator)

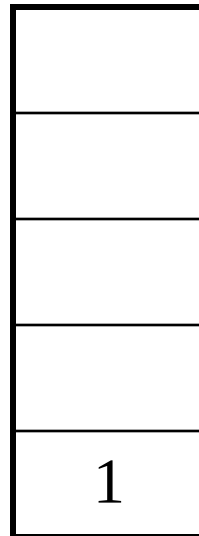


Finally, a '*' is seen and 48 and 6 are popped; the result, $6 * 48 = 288$, is pushed.



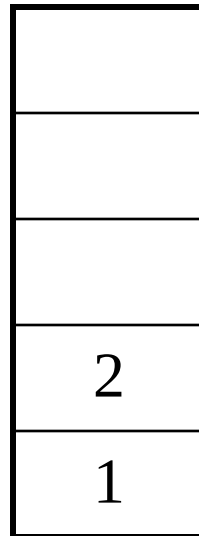
Example 2 (Calculator)

- Input: 1 2 3 + 4 * + 5 +
- Push(1)



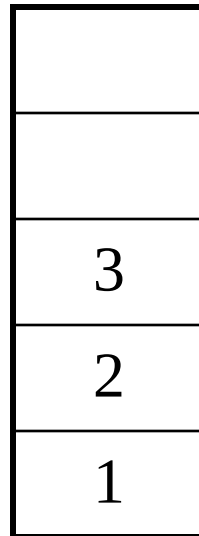
Example 2 (Calculator)

- Input: 2 3 + 4 * + 5 +
- Push(2)



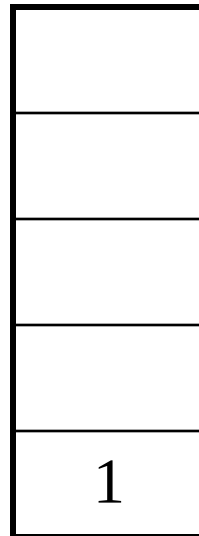
Example 2 (Calculator)

- Input: $3 + 4 * + 5 +$
- Push(3)



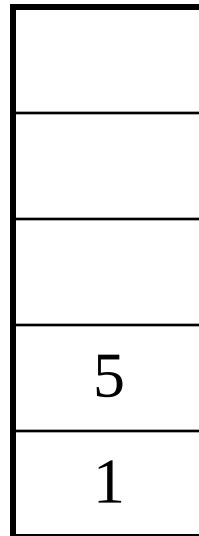
Example 2 (Calculator)

- Input: $+ 4 * + 5 +$
- $\text{Pop()} == 3$
- $\text{Pop()} == 2$



Example 2 (Calculator)

- Input: $+ 4 * + 5 +$
- Push($2 + 3$)



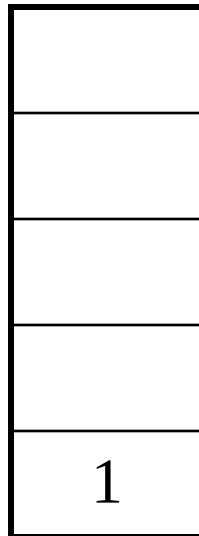
Example 2 (Calculator)

- Input: $4 * + 5 +$
- Push(4)

4
5
1

Example 2 (Calculator)

- Input: * + 5 +
- Pop() == 4
- Pop() == 5



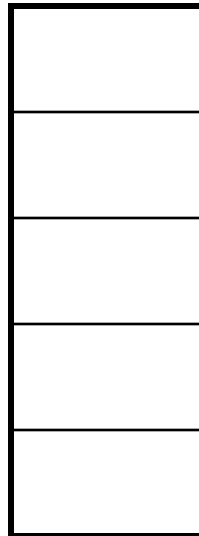
Example 2 (Calculator)

- Input: $*$ $+$ 5 $+$
- Push($5 * 4$)

20
1

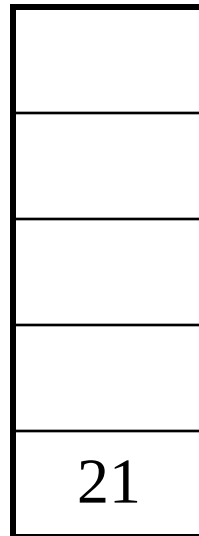
Example 2 (Calculator)

- Input: + 5 +
- Pop() == 20
- Pop() == 1



Example 2 (Calculator)

- Input: + 5 +
- Push(1 + 20)



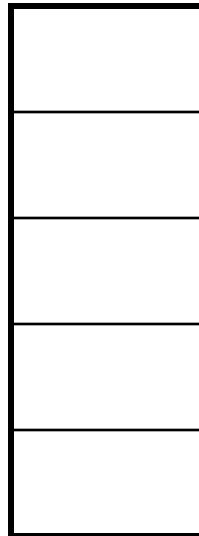
Example 2 (Calculator)

- Input: 5 +
- Push(5)

5
21

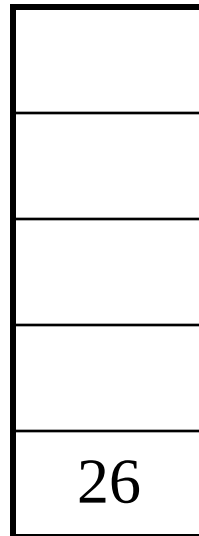
Example 2 (Calculator)

- Input: +
- Pop() == 21
- Pop() == 5



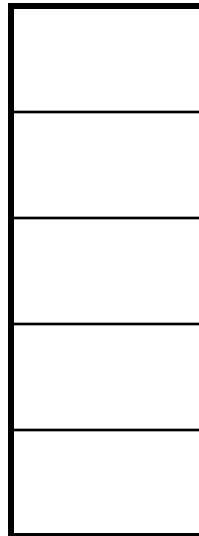
Example 2 (Calculator)

- Input: +
- Push($21 + 5$)



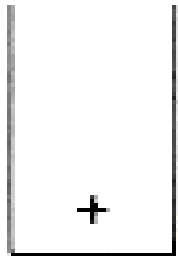
Example 2 (Calculator)

- Input:
- `Pop() == 26`



Example 1 (Infix to Postfix)

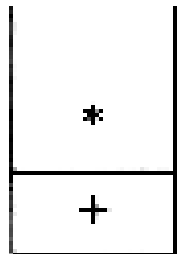
Infix/Inorder : $a + b * c + (d * e + f) * g$



Stack



Output



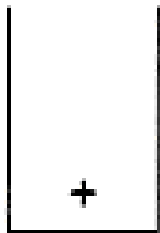
Stack



Output

Example 1 (Infix to Postfix)

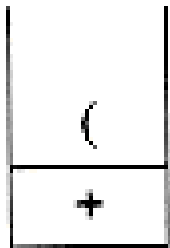
Infix/Inorder : $a + b * c + (d * e + f) * g$



Stack

a b c * +

Output



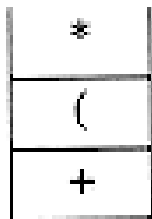
Stack

a b c * + d

Output

Example 1 (Infix to Postfix)

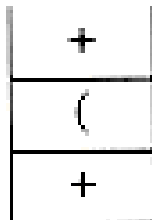
Infix/Inorder : $a + b * c + (d * e + f) * g$



Stack

a b c * + d e

Output



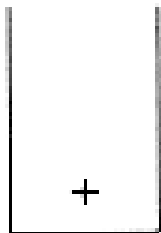
Stack

a b c * + d e * f

Output

Example 1 (Infix to Postfix)

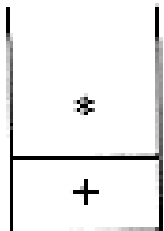
Infix/Inorder : $a + b * c + (d * e + f) * g$



Stack

$a b c * + d e * f +$

Output



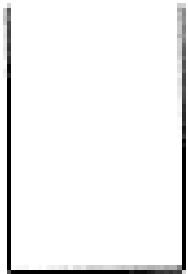
Stack

$a b c * + d e * f + g$

Output

Example 1 (Infix to Postfix)

Infix/Inorder : $a + b * c + (d * e + f) * g$



Stack

$a b c * + d e * f + g * +$

Output

Example 2 (Infix to Postfix)

$(((A + B) * (C - E)) / (F + G))$

- stack: <empty>
- output: []

Example 2 (Infix to Postfix)

$$((A + B) * (C - E)) / (F + G)$$

- stack: (
- output: []

Example 2 (Infix to Postfix)

$$(A + B) * (C - E) / (F + G)$$

- stack: ((
- output: []

Example 2 (Infix to Postfix)

$$A + B) * (C - E)) / (F + G))$$

- stack: (((
- output: []

Example 2 (Infix to Postfix)

$+ B) * (C - E)) / (F + G))$

- stack: (((
- output: [A]

Example 2 (Infix to Postfix)

$B) * (C - E) / (F + G)$

- stack: (((+
- output: [A]

Example 2 (Infix to Postfix)

) * (C - E)) / (F + G))

- stack: (((+
- output: [A B]

Example 2 (Infix to Postfix)

$$* (C - E)) / (F + G))$$

- stack: ((
- output: [A B +]

Example 2 (Infix to Postfix)

$$(C - E) / (F + G)$$

- stack: ((*
- output: [A B +]

Example 2 (Infix to Postfix)

$C - E)) / (F + G))$

- stack: ((* (
- output: [A B +]

Example 2 (Infix to Postfix)

- E)) / (F + G))

- stack: ((* (
- output: [A B + C]

Example 2 (Infix to Postfix)

$E)) / (F + G))$

- stack: ((* (-
- output: [A B + C]

Example 2 (Infix to Postfix)

$)) / (F + G))$

- stack: ((* (-
- output: [A B + C E]

Example 2 (Infix to Postfix)

) / (F + G))

- stack: ((*
- output: [A B + C E -]

Example 2 (Infix to Postfix)

$/(F + G))$

- stack: (
- output: [A B + C E - *]

Example 2 (Infix to Postfix)

$(F + G))$

- stack: (/
- output: [A B + C E - *]

Example 2 (Infix to Postfix)

$F + G))$

- stack: (/ (
- output: [A B + C E - *]

Example 2 (Infix to Postfix)

+ G))

- stack: (/ (
- output: [A B + C E - * F]

Example 2 (Infix to Postfix)

G))

- stack: (/ (+
- output: [A B + C E - * F]

Example 2 (Infix to Postfix)

))

- stack: (/ (+
- output: [A B + C E - * F G]

Example 2 (Infix to Postfix)

)

- stack: (/
- output: [A B + C E - * F G +]

Example 2 (Infix to Postfix)

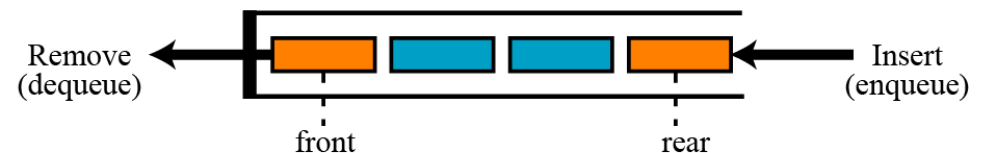
- stack: <empty>
- output: [A B + C E - * F G + /]

3.4 Queue ADT

- A **queue** is a linear list in which data can only be inserted at one end, called the **rear**, and deleted from the other end, called the **front**.
- A queue is a **First In First Out (FIFO)** structure.



A queue of people

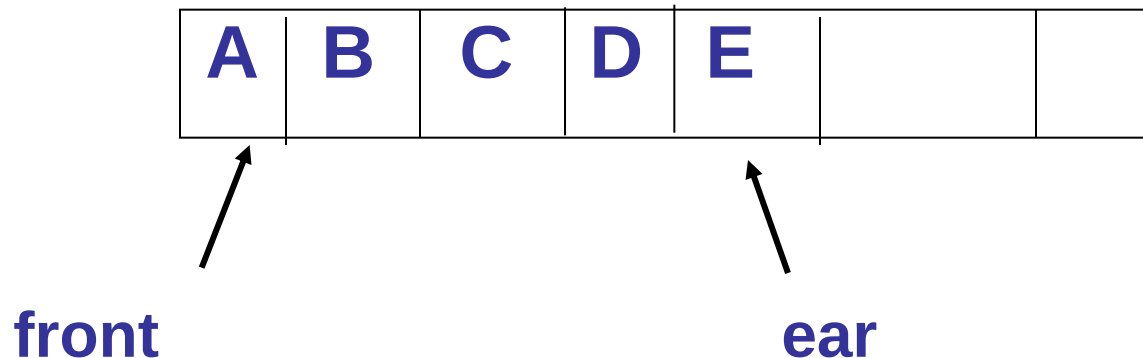


A computer queue

3.4 Queue ADT

- Main operations:
 - ❖ Delete item at front of the queue (Dequeue)
 - ❖ Insert item at rear of the queue (Enqueue)
 - ❖ Make empty

❖ ...



3.4 Applications of Queues

- Print jobs
- Computer networks
- OS
- Real-life waiting lines

3.4 Queue ADT -Array implementation

```
struct queue_record
{
    unsigned int q_max_size; /* Maximum # of elements */
    /* until Q is full */
    unsigned int q_front;
    unsigned int q_rear;
    unsigned int q_size; /* Current # of elements in Q */
    element_type *q_array;
};
```

```
typedef struct queue_record * QUEUE;
```

Figure 3.57 Type declarations for queue--array implementation

3.4 Queue ADT -Is Empty?

```
int is_empty( QUEUE Q )  
{  
    return( Q->q_size == 0 );  
}
```

Figure 3.58 Routine to test whether a queue is empty-array implementation

3.4 Queue ADT - Make NULL

```
void make_null ( QUEUE Q )  
{  
    Q->q_size = 0;  
    Q->q_front = 1;  
    Q->q_rear = 0;  
}
```

Figure 3.59 Routine to make an empty queue-array implementation

3.4 Queue ADT - Enqueue

```
void enqueue( element_type x, QUEUE Q )
{
    if( is_full( Q ) )
        error("Full queue");
    else
    {
        Q->q_size++;
        Q->q_rear = succ( Q->q_rear, Q );
        Q->q_array[ Q->q_rear ] = x;
    }
}
```

Figure 3.60 Routines to enqueue-array implementation

3.4 Queue ADT - Enqueue - Succ

```
unsigned int succ( unsigned int value, QUEUE Q )  
{  
    if( ++value == Q->q_max_size )  
        value = 0;  
    return value;  
}
```

3.4 Queue ADT- Dequeue

```
Void Dequeue( Queue Q )  
{  
    if( IsEmpty( Q ) )  
        Error( "Empty queue" );  
    else  
    {  
        Q->Size--;  
        Q->Front = Succ( Q->Front, Q );  
    }  
}
```

3.4 Queue ADT - No Array

```
typedef struct node *node_ptr;  
  
struct node  
{  
    element_type element;  
    int front;  
    int rear;  
    node_ptr next;  
};  
typedef node_ptr LIST;  
typedef node_ptr position;
```

Algorithm Analysis And Data Structures

Questions and Answers

Thank you

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)